We've added the sorbet and sorbet-runtime gems, which manage static and runtime checking, and the tapioca gem, which manages external type information, which Sorbet puts in .rbi files. The Sorbet gem gives us a runtime command called srb which we'll do type checking with momentarily.

To initialize Sorbet, we need to run tapioca init. This may take a bit and will generate a lot of output:

```
$ bundle exec tapioca init
```

This should generate a folder called sorbet. It contains a top level sorbet/config file, a subdirectory called rbi that has RBI type definitions for all the gems in the project, and a subdirectory called tapioca that has configurations and settings for the tapioca tool.

The next step is to run static type checking analysis, the command is bundle exec srb tc. In theory, because we've added no type checking this should pass, but in practice, if we've done anything that Sorbet doesn't like it will get flagged here. This might include dynamic references to constants or included modules that would make static analysis difficult.

Let's see what we get:

```
bundle exec srb tc
No errors! Great job.
```

That's encouraging, and a testament to the Aaagmnr gem being somewhat simple.

Now let's add the type checks.

Adding Type Checks

Type checks in Sorbet are written in the Ruby file, in—well, it's probably not quite accurate to say they are written in plain Ruby, but the Sorbet type checks are written as Ruby method calls in valid Ruby syntax. They do take a little getting used to.

Here's what the the Finder class looks like with Sorbet annotations added:

sorbet/aaagmnr/lib/aaagmnr/finder.rb

```
# typed: true

require "sorbet-runtime"

module Aaagmnr
  class Finder
    extend T::Sig

    sig { params(file_name: String).returns(Aaagmnr::Finder) }
    def self.from_file(file_name)
      new(File.readlines(file_name))
    end

    sig { params(dictionary_words: T::Array[String]).void }
    def initialize(dictionary_words)
      @signatures = T.let({}, T::Hash[String, T::Array[String]])
      dictionary_words.each do |line|
        word = line.chomp
        signature = signature_of(word)
        (@signatures[signature] ||= []) << word
      end
```